Data Structures

Topic

To understand the concept of **Linked Lists (Doubly & Circular)** through real-world applications such as a **Music Playlist Manager** and a **Multiplayer Board Game System**.

Content

This lab will cover two practical applications of linked lists:

1. **Movie Watchlist Manager (Doubly Linked List)**
2. **Playlist Manager (Doubly Linked List)** – Students will implement adding, deleting, navigating (next/previous), reversing (shuffle), and displaying songs in a playlist.
3. **Multiplayer Board Game (Circular Linked List)** – Students will implement adding/removing players, rotating turns, skipping turns, and ending the game when one player remains.

Task 1

Imagine you are working as a software developer for a **video streaming platform**. The company wants you to build a **Watchlist Manager** so users can organize movies they want to watch. Since users frequently move forward and backward between movies, you are required to implement this system using a **Doubly Linked List**.

Each node in the list represents a movie and stores the following information:

- **MovieID** (integer, unique identifier)
- **MovieName** (string, title of the movie)
- **Rating** (float, IMDb-style rating)

Your program must support the following operations:

1. Add Movie at End

Users should be able to add movies to the end of the watchlist. Each new movie must be properly linked in both directions.

2. Remove Movie by Name

Allow users to delete a movie by its name.

The movie may be:

- At the beginning
- In the middle
- At the end

3. Play Next &Play Previous

Users should be able to:

- Move to the **next movie**
- Move to the **previous movie**

4. Reverse Watchlist: Implement a function to **reverse the entire watchlist** using pointer manipulation (not by creating a new list).

5. Display Watchlist :Print all movies in order showing MovieID , Movie Name Rating.

1. Implement a function to **find the highest-rated movie in the list**

Task 2 :

Imagine you are hired as a software developer at a music streaming company. The company wants you to design a Playlist Manager that allows users to organize and control their music in a simple way. Since users often want to move back and forth between songs, you have been asked to implement this system using a Doubly Linked List. In this setup, every node will represent a song and store the following information:

- `SongID` (an integer for uniquely identifying the song)
- `SongName` (a string for the title of the song)
- `Duration` (a float representing the length of the song in minutes)

Your program should provide the following features:

- Add a new song at the end of the playlist. Users should be able to add unlimited songs, and each one must be linked correctly in both directions.
- Delete a song by name if the user wants to remove it from the playlist. This requires careful handling of links because the song could be at the beginning, middle, or end.
- Play Next and Play Previous – The user should be able to move forward to the next song or backward to the previous song seamlessly. This simulates pressing the “Next” or “Back” button in real music players.
- Shuffle (Reverse) Playlist – Users often like to shuffle their playlists. In this system, instead of random shuffling, you must implement a function that reverses the playlist order using the doubly linked list. This requires reassigning all next and previous pointers.
- Display the playlist – Show all songs with their ID, Name, and Duration in order.

Task 3

Imagine you are tasked with designing the backend system of a multiplayer digital board game (similar to *Ludo* or *Monopoly*). In this game, players take turns in a circular manner—once the last player finishes their turn, the first player plays again. To efficiently manage this rotation of turns, you must implement the system using a circular linked list where every node represents a player. Each node will store the following details:

- `PlayerID` (a unique integer for identifying each player)
- `Score` (an integer keeping track of the player's points during the game)

The game system must support the following operations:

1. Add a new player → Insert a player into the circular list. The game can start with any number of players, but new players can also join mid-game.
2. Remove a player if they quit → Delete their node from the circular linked list. This operation is tricky because the circular structure must remain intact (e.g., if the current player quits, the turn should move seamlessly to the next one).
3. Move to the next player's turn → Shift control to the next node in the circle, simulating how the game rotates turns.
4. Skip a player → Sometimes a penalty is applied (e.g., missing a turn). The system should be able to skip a player's turn and continue the cycle.
5. End game when only one player remains → As the game progresses, players may leave. The system should automatically detect when only one node remains in the list, declare that player as the winner, and terminate the game loop.